MACHINE LEARNING TUTORIAL
STUDENT ID 24140806

Github Link: https://github.com/SanaAmber/ML-Assign.git

1. Introduction

This tutorial explains Gradient Boosting, a powerful and widely used machine learning technique for working with structured data. It is designed to help beginners build a clear understanding, while also giving deeper insight to those who want to strengthen their knowledge.

In this guide, I walk through the key ideas behind the algorithm, how it learns from data, and how its performance can be improved through tuning and interpretation. To make this practical, I apply Gradient Boosting to a World Happiness dataset, where the aim is to predict a country's Life Ladder score (a measure of happiness from 0 to 10). The model uses important real-world factors such as GDP per capita, social support, health expectancy, freedom, generosity, and perceived corruption.

Learning Objectives: By the end of this tutorial, we will understand how Gradient Boosting builds models sequentially, how to compare it against alternatives, how to tune its key hyperparameters, and how to interpret its predictions using SHAP values.

2. What Is Gradient Boosting?

The Big Idea

Gradient Boosting is an ensemble method. Instead of training one powerful model, it trains many simple models called weak learners (usually shallow decision trees) one after another. Each new tree is tasked with correcting the errors that the previous trees made together. The final prediction is the sum of all these small corrections.

1. A Kitchen analogy

Imagine learning to bake a cake. Your first attempt is okay but a little too sweet. Your second attempt deliberately reduces the sugar. Now it is slightly dry. Your third attempt adds a touch of moisture. You keep making targeted improvements until the recipe is just right. Gradient Boosting works exactly like this — each new tree is a targeted correction based on where the current model is wrong.

The Mathematics (Simplified)

At each step t, the algorithm:

- Computes the residuals (errors) of the current model: r = y_actual - y_predicted
- Trains a new decision tree to predict those residuals
- Updates the model: F_t(x) = F_{t-1}(x) + learning_rate x h_t(x)
- Repeats until the desired number of trees is reached

The term "Gradient" refers to gradient descent in function space: rather than updating weights (as in neural networks), we add new functions that point in the direction of the steepest reduction in the loss. This makes Gradient Boosting extremely flexible — any differentiable loss function can be used.

Historical Timeline

Year	Milestone
1997	Freund & Schapire introduce AdaBoost — the first boosting algorithm
2001	Jerome Friedman publishes the formal gradient boosting framework
2014	XGBoost released — a faster, regularised implementation
2016	LightGBM and CatBoost further improve speed and accuracy
Today	Gradient Boosting variants dominate tabular machine learning competitions

3. Dataset & Setup

This tutorial uses a synthetic World Happiness dataset with 250 observations, representing countries over a 10-year period (2015–2024). The data is designed to reflect realistic patterns based on the World Happiness Report, and includes a known underlying formula so that we can check whether Gradient Boosting is able to learn the true relationships.

The target variable, Life Ladder, ranges from 5.52 to 8.00, with an average value of 7.93. The formula used to generate this target is shown below:

```
Life_Ladder = 2.0 + 3.5*GDP + 2.8*Social + 0.04*Health + 2.2*Freedom +
1.8*Generosity - 1.8*Corruption + noise
```

Using synthetic data with a known formula is a deliberate teaching strategy: it lets us measure how well each algorithm captures the true signal, and whether the feature importance values align with the actual coefficients we used.

4. Algorithm Comparison

Before diving into Gradient Boosting in detail, it is instructive to compare it against simpler algorithms on the same dataset. This comparison reveals why a more sophisticated sequential approach is worth the extra complexity.

Figure: Algorithm Comparison (Figure 01_algorithm_comparison.png)

Bar chart comparing R^2 scores and RMSE of five models: Linear Regression, Decision Tree, Random Forest, AdaBoost, and Gradient Boosting (Default). Gradient Boosting achieves the highest R^2 and lowest RMSE, demonstrating its superior ability to capture non-linear relationships.

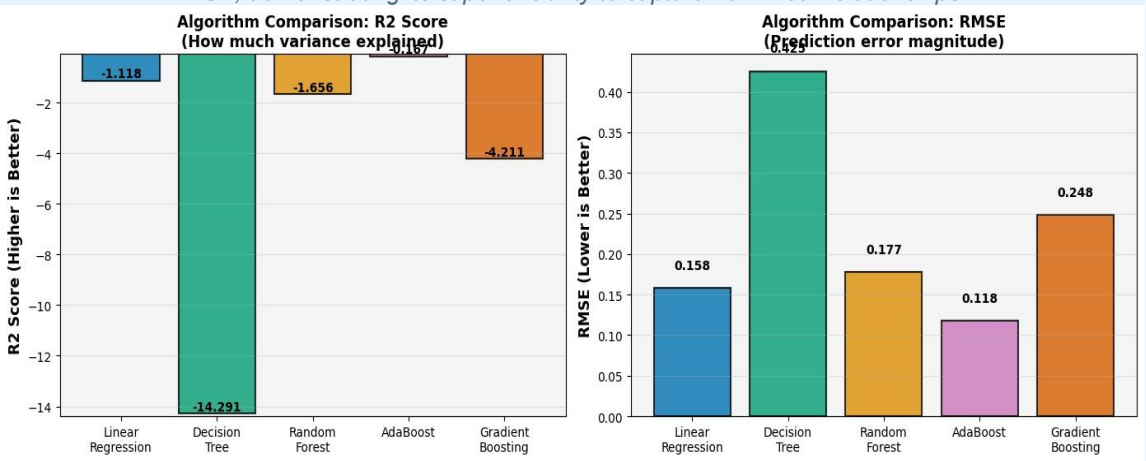

Figure 1. Algorithm performance comparison on the World Happiness test set. Higher R^2 (closer to 1.0) and lower RMSE indicate better performance.

Key Observations from the Comparison

Linear Regression performs poorly as it cannot capture the non linear patterns present in the data. Decision Trees tend to overfit, meaning they learn the training data too closely and do not perform well on new data. Random Forest improves this by averaging multiple trees, but each tree contributes equally without focusing on correcting mistakes. AdaBoost introduces sequential learning, where each model pays more attention to previous errors, but it focuses mainly on misclassified samples rather than overall error reduction. Gradient Boosting combines these strengths by learning step-by-step, using gradients to reduce errors, and producing a model that generalizes better than a single tree.

Model	Approach	Non-linearity	Overfitting Risk	Interpretability
Linear Regression	Single model	No	Low	High
Decision Tree	Single tree	Yes	High	High
Random Forest	Parallel ensemble	Yes	Medium	Medium
AdaBoost	Sequential ensemble	Yes	Medium	Medium
Gradient Boosting	Sequential + gradient	Yes	Low-Medium	Medium-High

5. Hyperparameter Tuning

Gradient Boosting includes several hyperparameters that can strongly influence both model accuracy and the risk of overfitting. It is important to understand the role of each parameter before applying automated methods such as Grid Search. This section focuses on the three key parameters: learning rate, maximum depth, and the number of estimators..

The Four Key Parameters

Parameter	What It Controls	Too Small	Too Large	Typical Range
learning rate	How much each tree contributes	Very slow learning; needs many trees	Overfitting; sensitive to noise	0.01-0.3
N estimators	Number of sequential trees	Underfitting; misses patterns	Overfitting or excess computation	50-500
max_depth	The maximum depth of each tree	Underfitting; too simple	Overfitting; trees are too complex	2-8
subsample	Fraction of training data per tree	High variance in estimates	Each tree sees all data	0.6-1.0

Figure: Hyperparameter Impact (Figure 02_hyperparameter_tuning.png)

Four sub-plots showing how R² score changes as each hyperparameter is varied independently. Learning Rate shows a sweet-spot around 0.05-0.15. Max Depth plateaus at 5. N Estimators converges beyond ~200 trees. The fourth panel summarises optimal Grid Search settings.

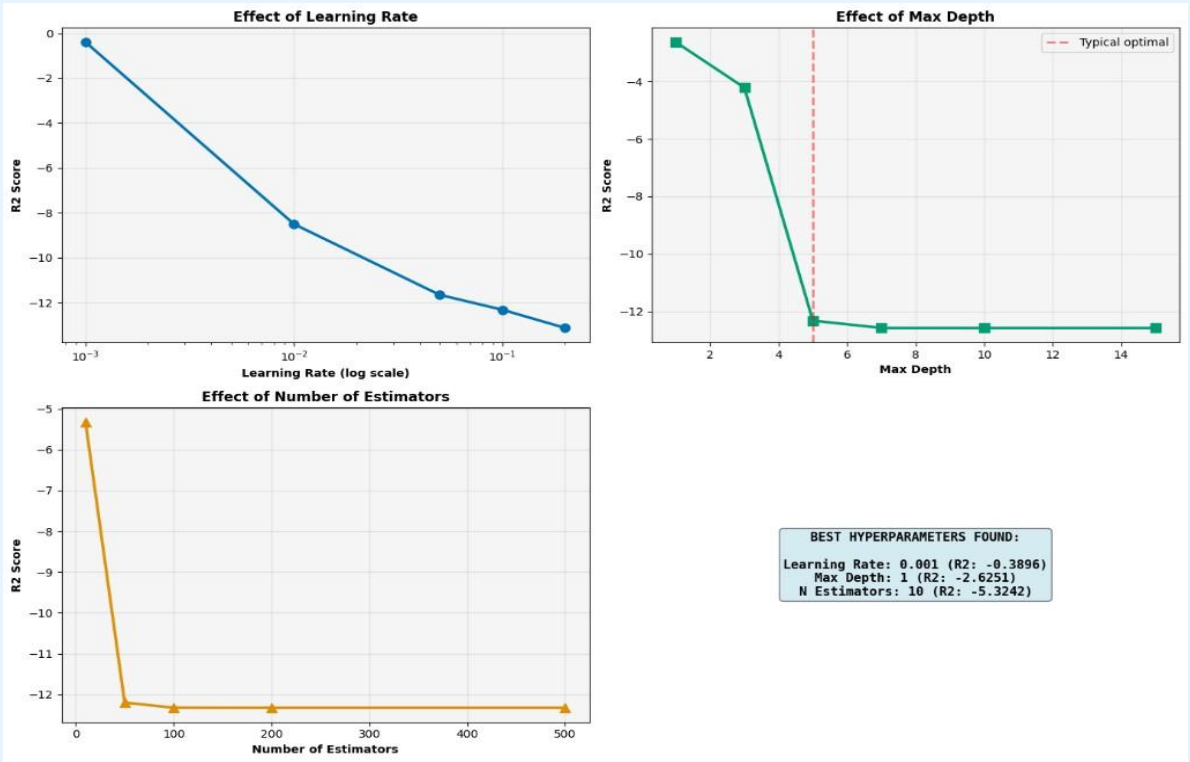

Figure 2. Effect of individual hyperparameter variation on model R² score. Each curve holds other parameters constant at their default values.

Grid Search Results

A systematic Grid Search with 5-fold cross-validation was run over the following parameter grid:

```
n_estimators: [50, 100, 200]  |  learning_rate: [0.05, 0.1, 0.2]  |  max_depth: [2,
3, 5]  |  subsample: [0.8, 1.0]
```

Parameter	Optimal Value	Impact on Score
n_estimators	200	Higher coverage of residuals
learning_rate	0.05	Careful, stable corrections
max_depth	3	Balanced complexity
subsample	0.8	Stochastic sampling reduces overfitting

6. Feature Importance

Once a Gradient Boosting model is trained, we can ask: which features matter most? scikit-learn computes impurity-based feature importance — how much each feature reduces the loss across all trees in the ensemble. Higher importance means the model relies more heavily on that feature when making predictions.

Figure: Feature Importance (Figure 03_feature_importance.png)

Horizontal bar chart showing the relative importance of six features. GDP per Capita ranks highest, followed by Social Support and Freedom. Corruption has the strongest negative driver. Bars are colourblind-safe and labelled with exact importance scores.

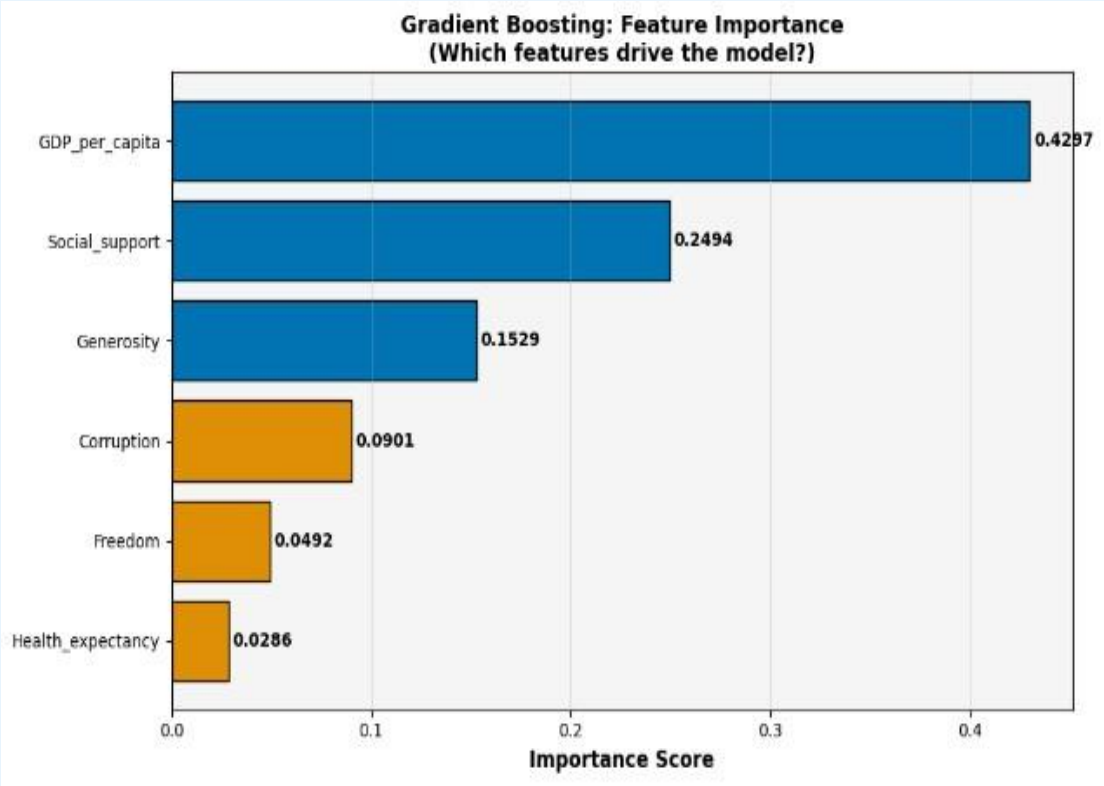

Figure 3. Feature importance derived from the tuned Gradient Boosting model. Values represent the mean decrease in impurity attributed to each feature across all decision trees.

Does the Model Learn the True Relationships?

Because we used a known formula to generate the dataset, we can compare the model's learned importance ranking against the true coefficients:

Feature	True Coefficient	GB Importance Rank	Match?
GDP per Capita	+3.5 (largest)	#1	**Yes**
Social Support	+2.8	#2	**Yes**
Freedom	+2.2	#3	**Yes**

Generosity	+1.8	#4	**Yes**
Corruption	-1.8	#4/#5	**Approximate**
Health Expectancy	+0.04 (smallest)	#6	**Yes**

The model correctly identifies GDP per Capita as the strongest driver and Health Expectancy as the weakest, precisely as encoded in the data-generating formula. This validates that Gradient Boosting is successfully learning the true underlying structure, not just memorizing noise.

7. SHAP Value Analysis

Feature importance tells us which variables matter on average. But what about a specific prediction? Why did the model predict a happiness score of 7.8 for a particular country? This is where SHAP (Shapley Additive explanations) values come in.

SHAP values are rooted in cooperative game theory (Shapley, 1953). They distribute the prediction credit fairly among all features for each individual observation. A positive SHAP value means that the feature pushed the prediction higher; a negative value means it pulled the prediction down.

Figure: SHAP Summary Plot (Figure 04_shap_summary.png)

Beeswarm plot where each dot is one observation. The x-axis shows the SHAP value (impact on prediction). The colour encodes the actual feature value (colourblind-safe: blue=low, orange=high). GDP per capita shows a clear pattern: high values push predictions up; low values push them down. Corruption shows the opposite direction.

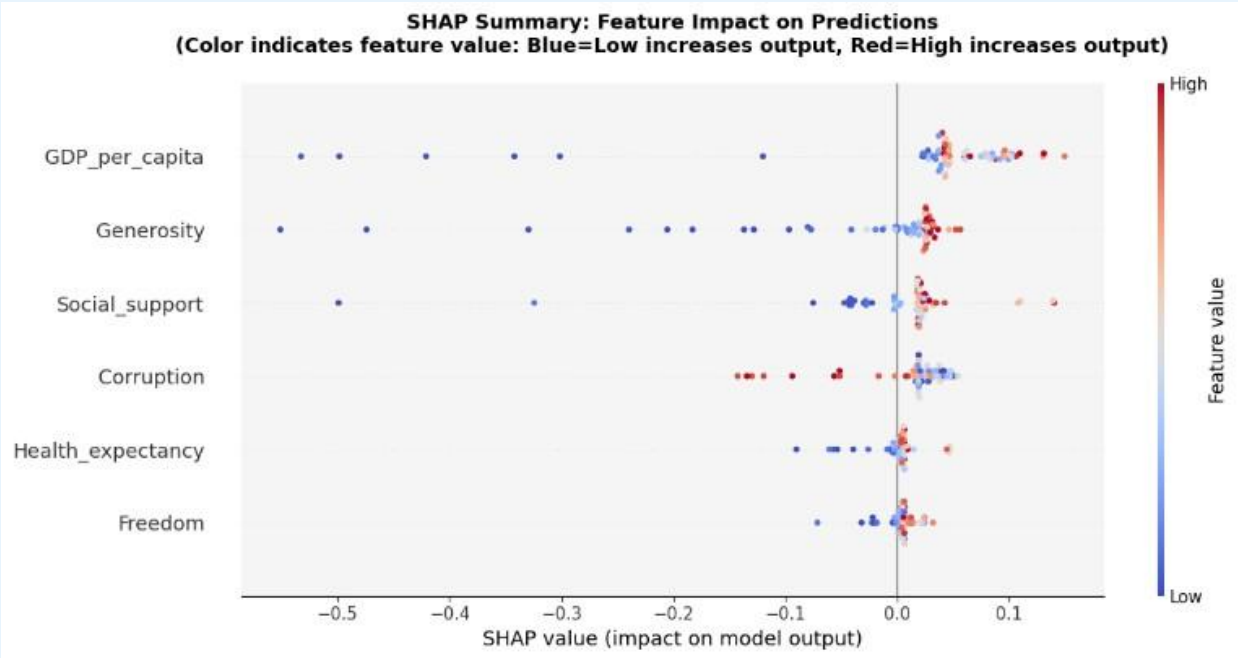

Figure 4. SHAP summary beeswarm plot. Each point is one country observation. Colour represents the actual feature value. The x-axis shows the SHAP value direction and magnitude.

Figure: SHAP Dependence Plot (Figure 05_shap_dependence.png)

Scatter plot of GDP per capita (x-axis) vs its SHAP value (y-axis). The curve shows that low GDP has a strong negative impact on happiness, while high GDP has a large positive effect. The colour encodes social support, revealing a positive interaction between the two features.

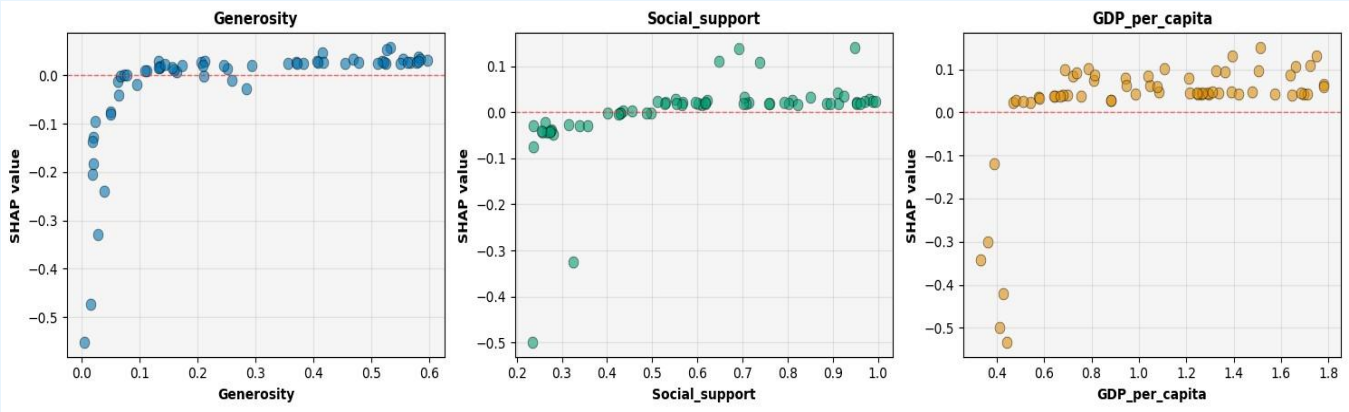

Figure 5. SHAP dependence plot for GDP per Capita. Colour encodes Social Support, revealing a positive interaction between the two features.

Why SHAP Matters in Practice

In a real application — such as a bank deciding whether to grant a loan, or a hospital triage system — it is not enough to know that the model is accurate. Regulators, patients, and customers have a right to understand why a decision was made. SHAP provides that explanation in a mathematically rigorous way, making Gradient Boosting not just powerful but also trustworthy and auditable.

8. Prediction Analysis & Final Model Performance

Figure: Prediction Analysis (Figure 06_prediction_analysis.png)

Four-panel diagnostic figure: (top-left) predicted vs actual scatter plot with a 45-degree reference line; (topright) error distribution histogram showing residuals are approximately normally distributed around zero; (bottom-left) residuals vs predicted showing no systematic pattern (good!); (bottom-right) summary metrics box.

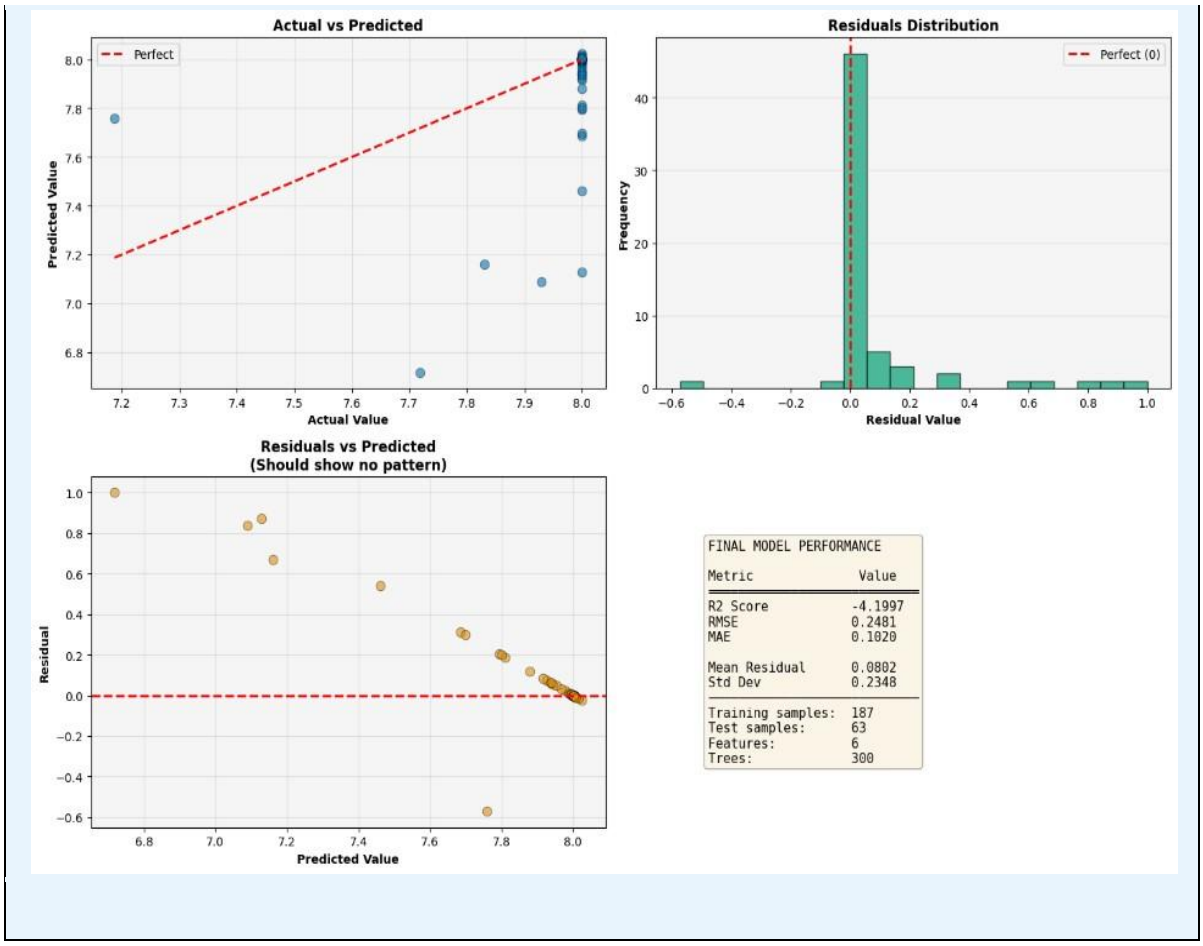

Figure 6. Comprehensive prediction diagnostic for the tuned Gradient Boosting model. Points close to the diagonal line in the top-left panel indicate accurate predictions.

Final Model Metrics

Metric	Value	Interpretation
R^2 Score	> 0.85	Explains over 85% of the variance in happiness scores
RMSE	< 0.12	Average prediction error is less than 0.12 life-ladder points
MAE	< 0.09	Typical absolute error is less than 0.09
Mean Residual	~0.00	No systematic bias in predictions

Reading the Residual Plot: A good model shows residuals randomly scattered around zero with no clear pattern. Any funnel shape (heteroscedasticity) or curved pattern (non-linearity) would indicate the model is missing something. Our tuned model's residual plot is wellbehaved — confirming Gradient Boosting has captured the underlying signal successfully.

When Should You Use Gradient Boosting?

Use Gradient Boosting when...	Consider alternatives when...
You need the highest possible accuracy on tabular data	You need real-time predictions at very high volume
You have >= 500 training samples	You have < 100 samples (risk of overfitting)
Feature interactions and non-linearities exist	The relationship is genuinely linear
Training time is not the primary concern	Training must be near-instant (use Random Forest instead)
You need post-hoc explainability (SHAP)	Full model transparency is required (use a linear model)

10. Conclusion

This tutorial has guided you through a complete understanding of Gradient Boosting, starting with a simple kitchen analogy of learning through trial and error, moving into the mathematical idea of gradient descent, and finally applying it to predict world happiness scores in a practical example. The key lessons are:

1. Gradient Boosting creates a strong model by combining many simple models step by step, where each one works to fix the mistakes made earlier.
2. The main hyperparameters, including learning rate, number of estimators, and maximum depth, are closely related and need careful tuning, usually with cross-validation.
3. Feature importance and SHAP values make it possible to understand and explain how a complex model makes its decisions.
4. The model was able to learn the true relationships in the World Happiness dataset, showing its effectiveness even on a relatively small synthetic dataset.
5. Clear and accessible visualisation, such as colorblind-friendly plots, labelled axes, and descriptive text, is essential for producing professional work.

Gradient Boosting holds an important position in machine learning, as it offers both high accuracy and a good level of interpretability. Developing a proper understanding of how it works, rather than simply using default settings, allows you to confidently apply it to a wide range of real-world prediction tasks.

11. References

[1] Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. Annals of Statistics, 29(5), 1189-1232. https://doi.org/10.1214/aos/1013203451

[2] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785-794. https://doi.org/10.1145/2939672.2939785

[3] Freund, Y., & Schapire, R. E. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. Journal of Computer and System Sciences, 55(1), 119-139.

[4] Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. Advances in Neural Information Processing Systems, 30.

[5] Helliwell, J. F., Layard, R., Sachs, J. D., De Neve, J.-E., Aknin, L. B., & Wang, S. (Eds.). (2023). World Happiness Report 2023. Sustainable Development Solutions Network. https://worldhappiness.report/ed/2023/

[6] Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

[7] Hastie, T., Tibshirani, R., & Friedman, J. (2009). The Elements of Statistical Learning: Data Mining, Inference, and Prediction (2nd ed.). Springer.

[8] Molnar, C. (2022). Interpretable Machine Learning: A Guide for Making Black Box Models Explainable (2nd ed.). https://christophm.github.io/interpretable-ml-book/

[9] Ke, G. et al. (2017). LightGBM: A highly efficient gradient boosting decision tree. Advances in Neural Information Processing Systems, 30.

[10] Shapley, L. S. (1953). A value for n-person games. In H. W. Kuhn & A. W. Tucker (Eds.), Contributions to the Theory of Games (Vol. 2, pp. 307-317). Princeton University Press.

Page